

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

Table 1: Dataset split used for the experiments

Dataset Split	Number of Images
Training	2944
Validation	736
Test	3669

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:

Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Experimental Results

Four weight initialization methods were compared: Zero, Random, Xavier/Glorot and He initialization. Each model was trained for 5 epochs using the same training and validation data.

Table 2: Comparison of weight initialization methods

Initialization	Final Training Loss	Best Validation Accuracy (%)	Training Time (s)
Zero	0.0905	91.30	41.96
Random	0.1069	90.35	41.71
Xavier/Glorot	0.0977	91.17	41.40
He	0.0988	91.58	41.91

Observation: All four initialization methods showed a clear decrease in training loss over the five epochs. He initialization achieved the highest validation accuracy of 91.58%, followed by Zero initialization at 91.30%. Random initialization gave the lowest validation accuracy among the four methods at 90.35%.

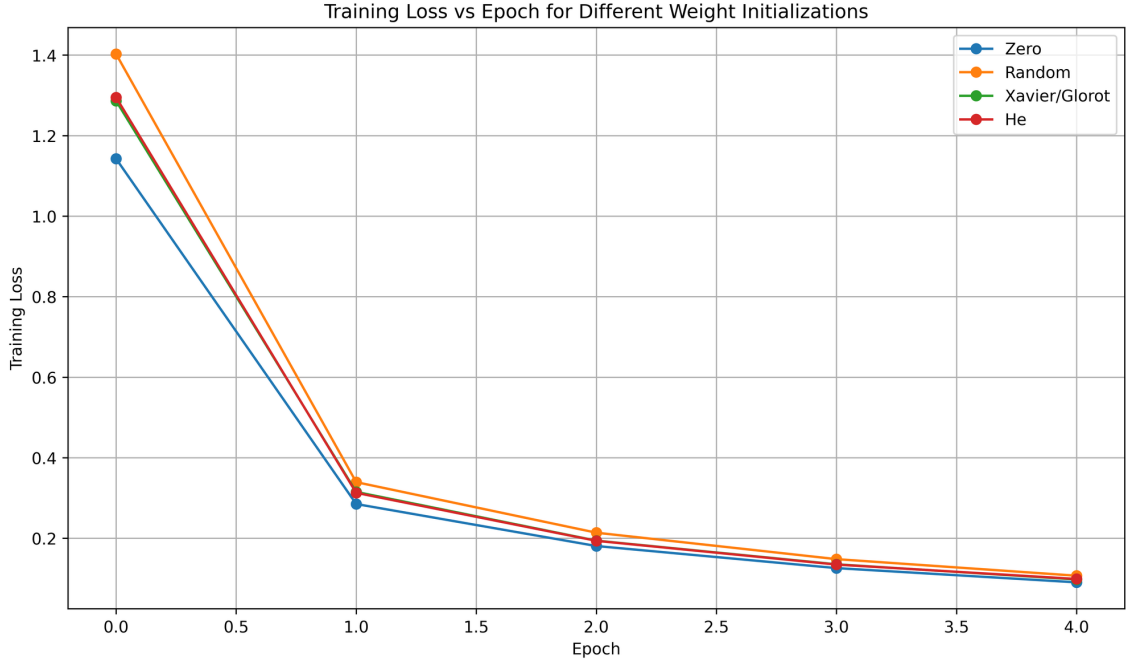


Figure 1: Training Loss vs. Epoch for Different Weight Initializations

Inference: The training loss decreases steadily for all four initialization methods over the five epochs. The final losses are close, ranging from 0.0905 for Zero initialization to 0.1069 for Random initialization, showing that the choice of initialization had only a small effect on training loss in this setup.

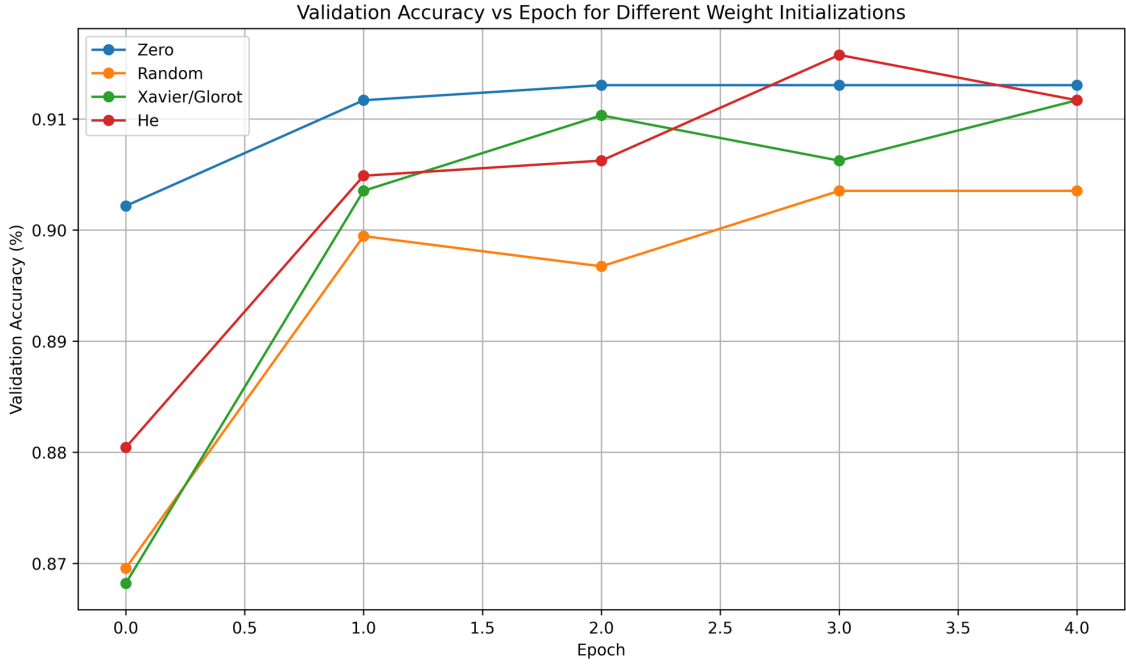


Figure 2: Validation Accuracy vs. Epoch for Different Weight Initializations

Inference: Validation accuracy improves quickly during the first few epochs and then starts to level off. He initialization reaches the highest validation accuracy of 91.58%, although its accuracy drops slightly in the final epoch. This indicates that He performed best overall, but the difference from Zero and Xavier/Glorot initialization was small.

Result: Among the four methods tested, He initialization gave the best validation performance and was therefore selected as the best initialization method for the subsequent experiments.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Experimental Results

Four cases were compared to study the effect of regularization on training and validation performance. All models were trained for 5 epochs using the same dataset and basic training setup.

Table 3: Comparison of regularization techniques

Regularization	Final Training Loss	Best Validation Accuracy (%)	Training Time (s)
No Regularization	0.0322	90.49	42.77
L2 Regularization	0.2394	90.90	42.48
Dropout	0.2584	89.95	44.18
Batch Normalization	0.0384	91.17	42.66

Observation: The model without regularization reached a very high training accuracy by the final epoch, but its validation accuracy remained lower. L2 regularization gave a slightly better validation result, while Dropout showed a slower increase in training accuracy. Batch Normalization achieved the highest validation accuracy of 91.17% among the four methods.

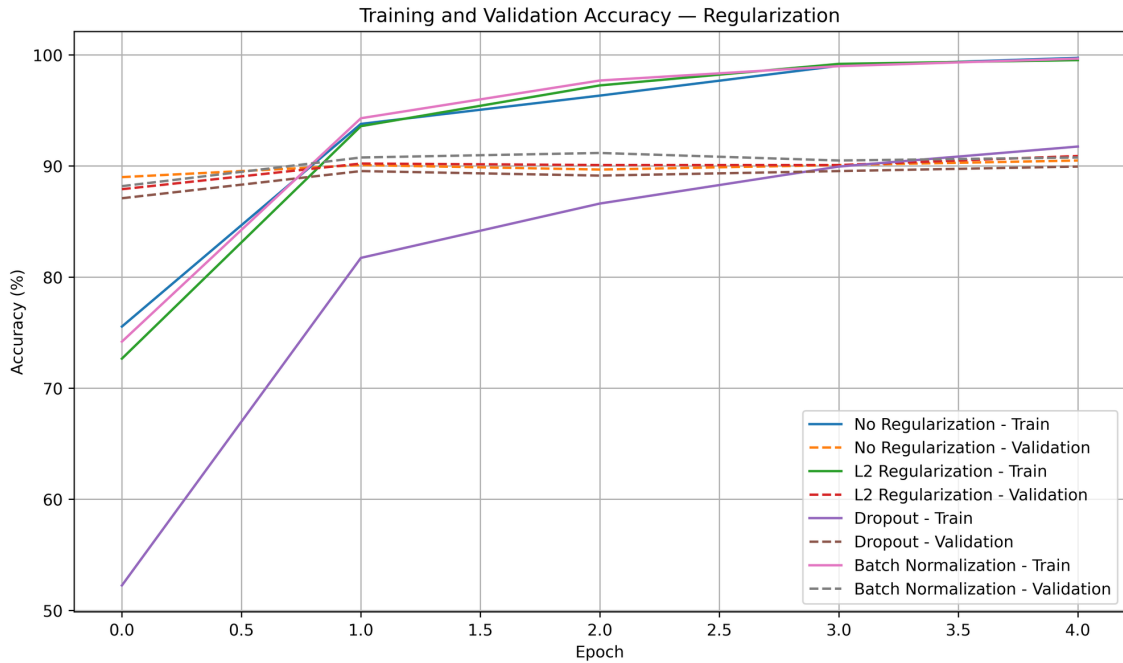


Figure 3: Training and Validation Accuracy for Different Regularization Methods

Inference: Training accuracy increased for all four regularization settings, while the validation results were different. Batch Normalization achieved the highest validation accuracy of 91.17%, followed by L2 at 90.90%, No Regularization at 90.49% and Dropout at 89.95%. The gap between training and validation performance is more noticeable for the weaker regularization settings.

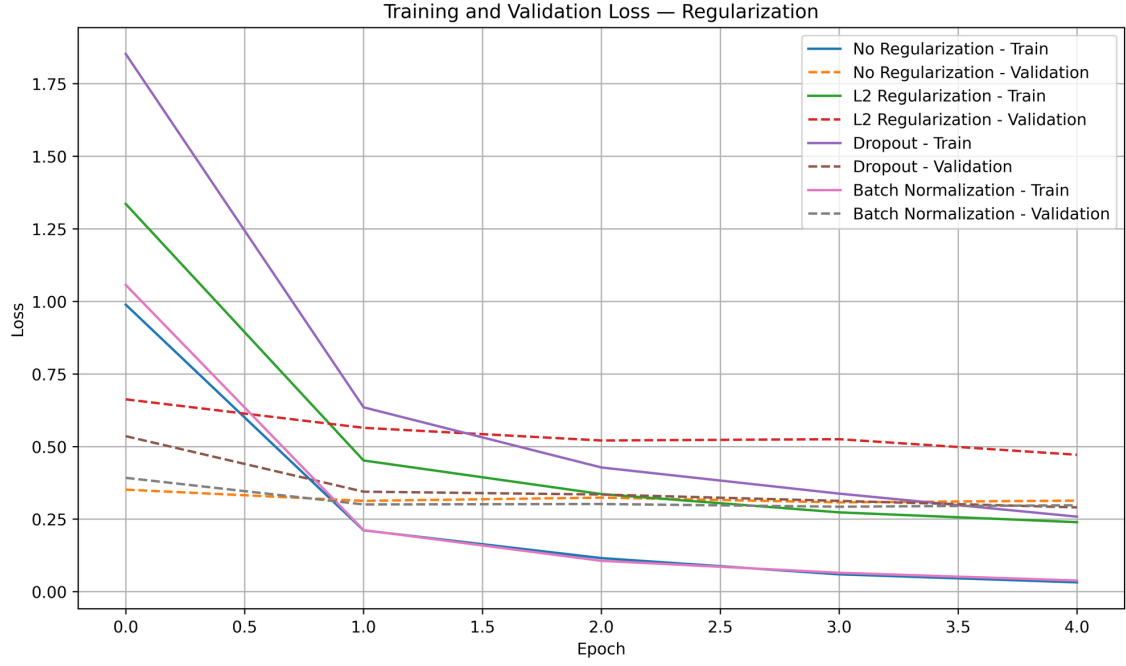


Figure 4: Training and Validation Loss for Different Regularization Methods

Inference: Training loss decreases consistently for all four methods. In the case of no regularization, the training loss becomes very small while the validation loss remains around the same level, which indicates a generalization gap. L2 and Dropout keep the training loss higher, while Batch Normalization reaches a low training loss together with the highest validation accuracy.

Result: Batch Normalization performed best in this comparison, with a validation accuracy of 91.17%. It was therefore considered the best regularization method based on validation accuracy.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Experimental Result

The effect of Batch Normalization was also compared experimentally by training the model with and without Batch Normalization while keeping the other settings unchanged.

Table 4: Effect of Batch Normalization on validation accuracy

Configuration	Best Validation Accuracy (%)
Without Batch Normalization	90.49
With Batch Normalization	91.17

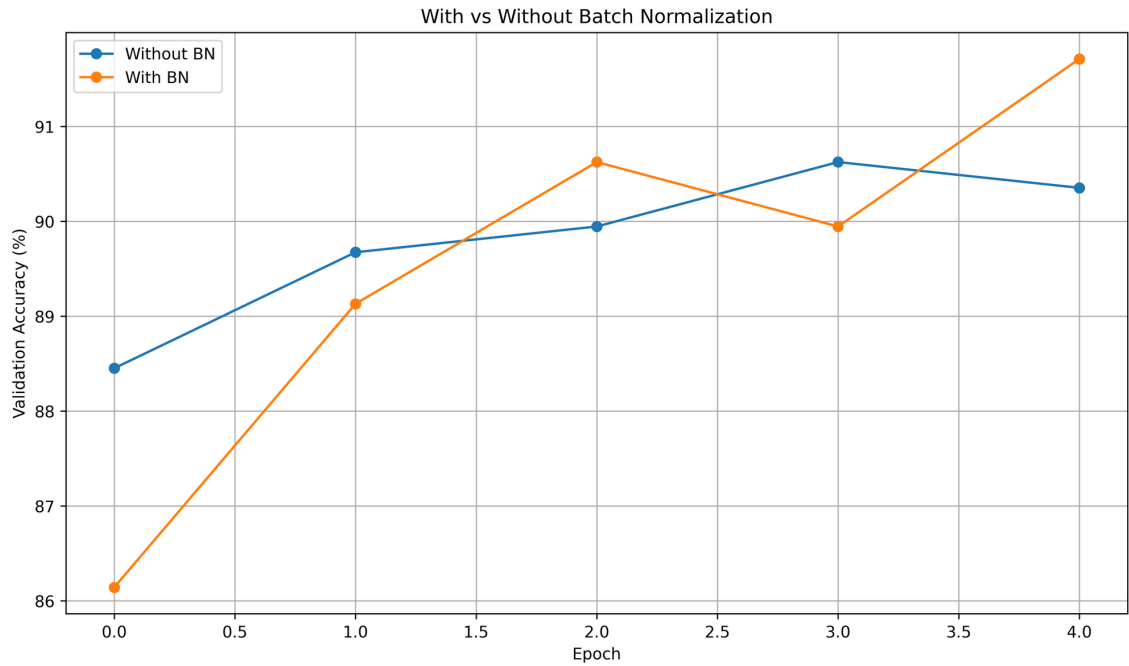


Figure 5: Validation Accuracy With and Without Batch Normalization

Inference: The model using Batch Normalization reaches a slightly higher validation accuracy and shows a more consistent improvement during training. The result suggests that normalizing the activations helped the model train more effectively for this dataset.

Result: Batch Normalization improved the best validation accuracy from 90.49% to 91.17%, giving an improvement of 0.68 percentage points in this experiment.

8. Optimization Algorithms

Table 5: Comparison of optimization algorithms

Optimizer	Final Loss	Best Validation Accuracy (%)	Best Epoch	Time (s)
SGD	3.0794	23.37	5	42.87
Momentum	0.4708	89.13	5	42.32
RMSProp	0.0464	89.81	5	42.35
Adam	0.0435	90.49	5	42.81

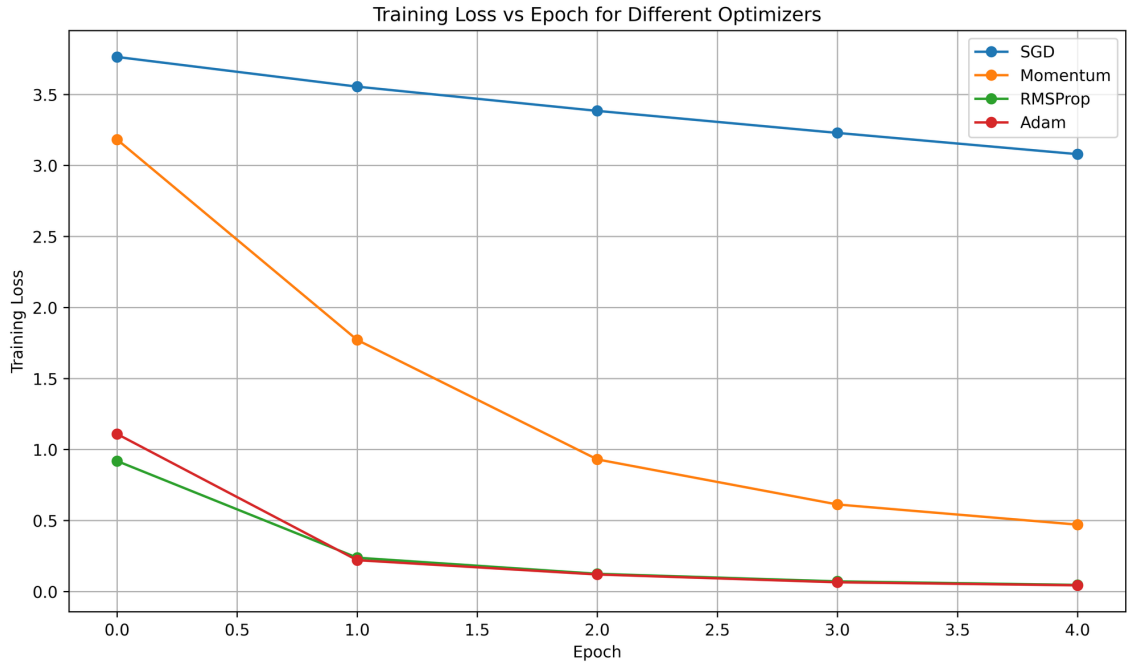


Figure 6: Training Loss vs. Epoch for Different Optimizers

Inference: The training loss decreases much more effectively for Momentum, RMSProp and Adam compared with basic SGD. SGD remains at a considerably higher loss throughout the experiment, indicating that it was not able to optimize the classifier effectively with the given learning rate. Adam and RMSProp reach the lowest losses by the end of training.

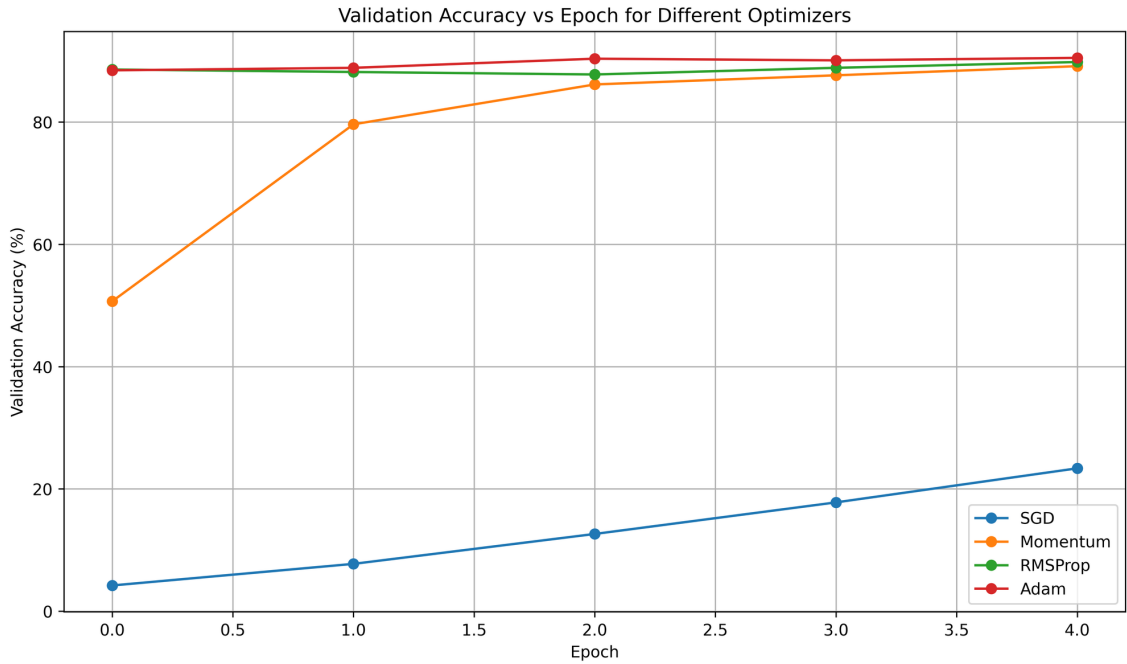


Figure 7: Validation Accuracy vs. Epoch for Different Optimizers

Inference: Adam gives the highest validation accuracy of 90.49%, followed by RMSProp at 89.81% and Momentum at 89.13%. SGD performs much worse, reaching only 23.37%. The results show that adaptive optimizers were much more suitable for this experiment than plain SGD.

Result: Among the four optimizers, Adam produced the best validation accuracy and the lowest final training loss. Therefore, Adam was selected as the optimizer for the subsequent hyperparameter experiments.

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Experimental Result

Two learning rates, 0.001 and 0.0001, were compared while keeping the other training settings unchanged.

Table 6: Effect of learning rate on validation accuracy

Learning Rate	Best Validation Accuracy (%)	Training Time (s)
0.001	91.44	43.85
0.0001	87.64	42.95

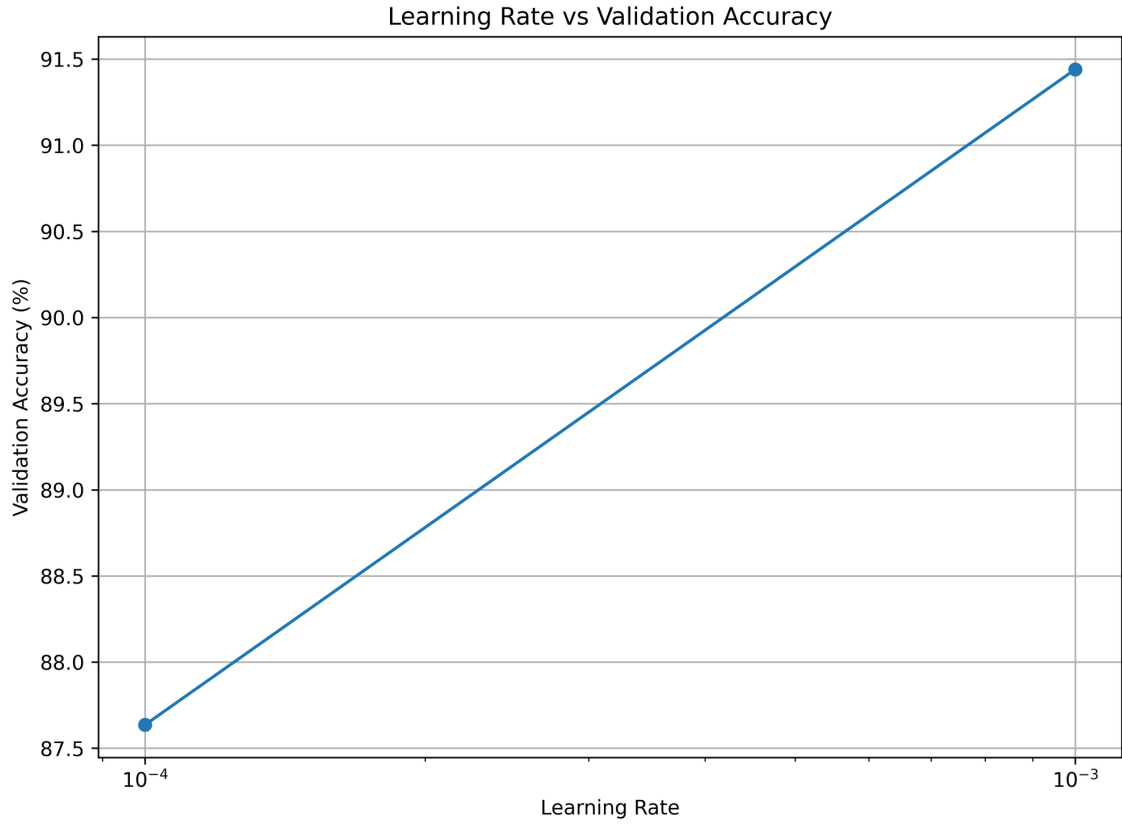


Figure 8: Learning Rate vs. Validation Accuracy

Observation: The learning rate of 0.001 performed clearly better than 0.0001. With 0.001, the validation accuracy reached 91.44% by the fifth epoch, whereas the lower learning rate reached only 87.64%. The smaller learning rate improved more slowly and did not reach the same level within the five training epochs.

Result: A learning rate of 0.001 was selected for the subsequent experiments.

Experimental Result

Batch sizes of 16, 32 and 64 were compared to study their effect on validation performance and training time.

Table 7: Effect of batch size on validation accuracy

Batch Size	Best Validation Accuracy (%)	Training Time (s)
16	90.62	52.58
32	90.90	43.04
64	91.30	56.38

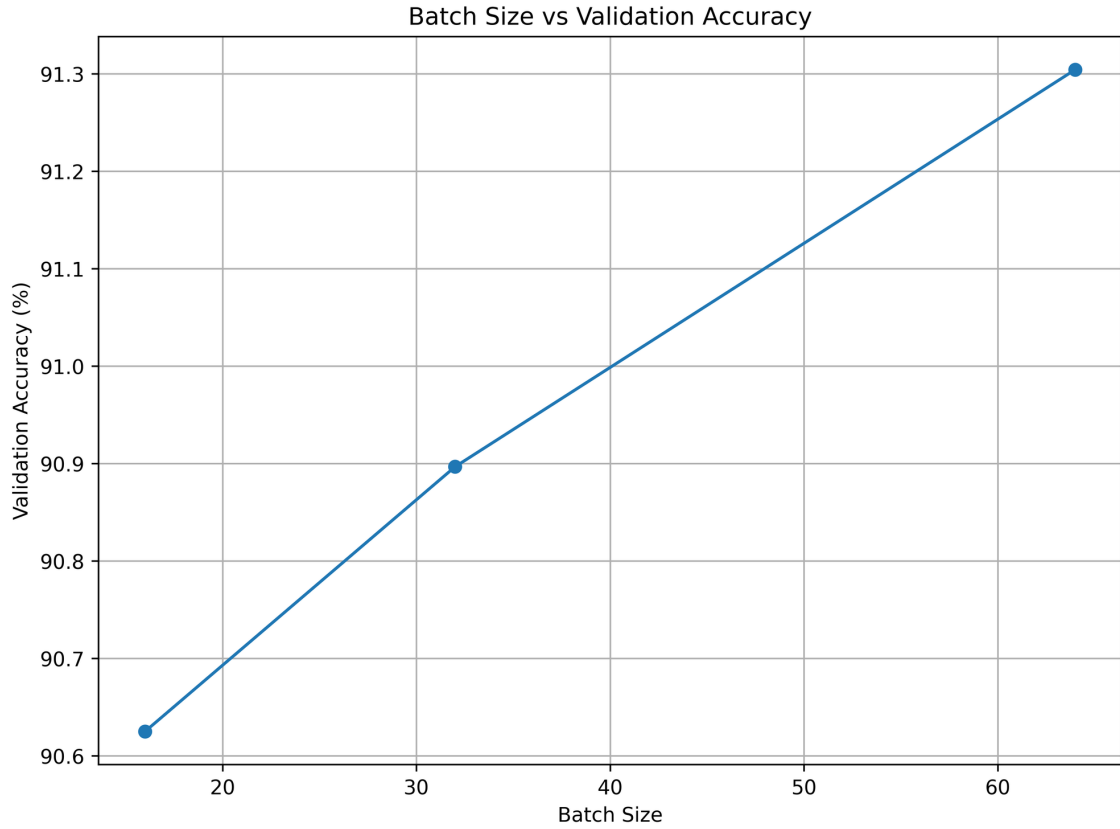


Figure 9: Batch Size vs. Validation Accuracy

Observation: Validation accuracy increased slightly as the batch size was increased. Batch size 64 achieved the highest validation accuracy of 91.30%, followed by 32 at 90.90% and 16 at 90.62%. However, batch size 32 had the lowest training time among the three in this particular run.

Result: Based on validation accuracy, a batch size of 64 was selected for the subsequent experiments.

Experimental Result

Dropout rates of 0, 0.25 and 0.5 were compared while keeping the other training settings unchanged.

Table 8: Effect of dropout rate on validation accuracy

Dropout Rate	Best Validation Accuracy (%)	Training Time (s)
0	89.67	42.43
0.25	91.30	43.01
0.50	90.22	42.90

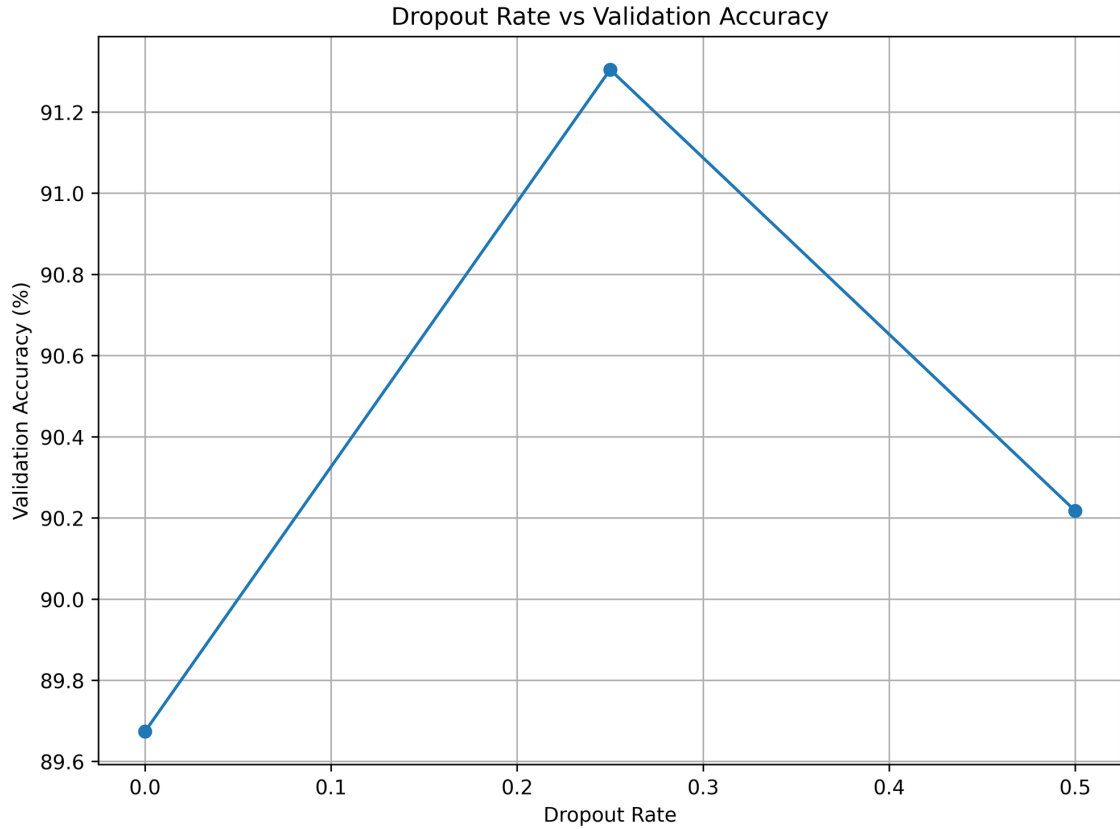


Figure 10: Dropout Rate vs. Validation Accuracy

Observation: The results show that a moderate amount of dropout worked better than either no dropout or a higher dropout rate. A dropout rate of 0.25 achieved the highest validation accuracy of 91.30%. Increasing the dropout rate further to 0.5 reduced the validation accuracy slightly to 90.22%.

Result: A dropout rate of 0.25 was selected as the best configuration among the tested dropout rates.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Experimental Results

Transfer learning was evaluated using two approaches: feature extraction, where the pretrained MobileNetV2 base was kept frozen, and fine-tuning, where selected layers of the pretrained network were allowed to train.

Table 9: Comparison of feature extraction and fine-tuning

Method	Best Validation Accuracy (%)	Final Validation Loss	Training Time (s)
Feature Extraction	90.49	0.3404	42.40
Fine-Tuning	89.95	0.3836	43.14

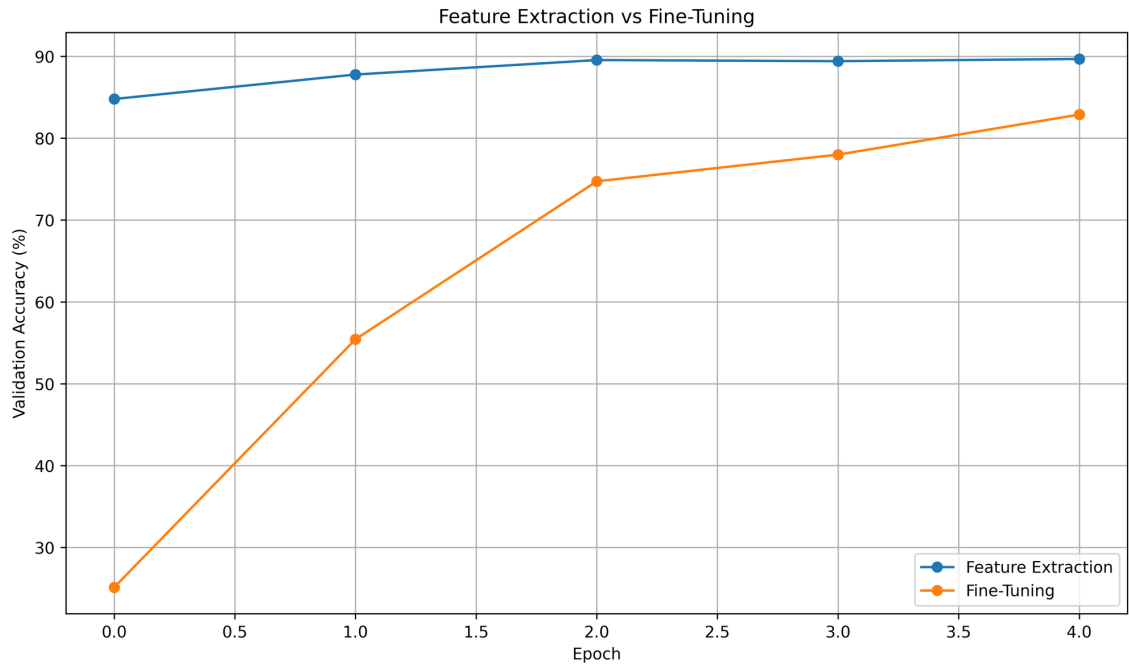


Figure 11: Feature Extraction vs. Fine-Tuning

Observation: Feature extraction performed slightly better than fine-tuning in this experiment. It reached a validation accuracy of 90.49%, while fine-tuning reached 89.95%. The difference is small, but the result shows that allowing the pretrained layers to update did not provide an improvement for this particular setup.

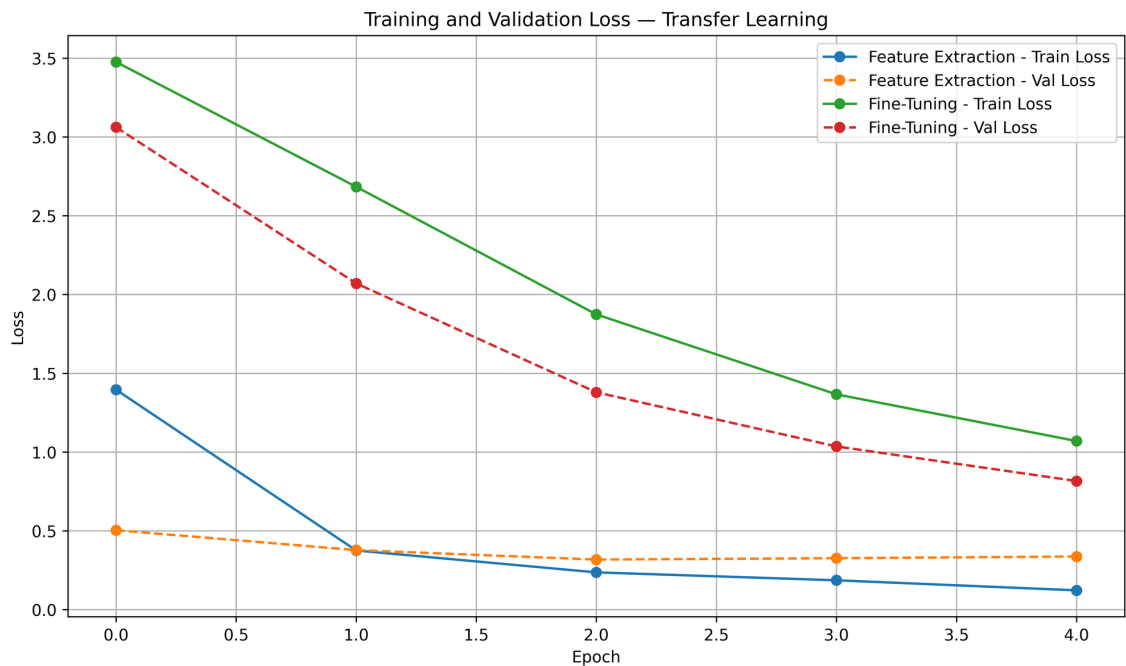


Figure 12: Training and Validation Loss for Transfer Learning

Inference: Both approaches show a decrease in training loss during training. Feature extraction maintains slightly better validation behaviour, while fine-tuning does not show an additional gain within the five epochs used. Since the pretrained MobileNetV2 features were already useful for this classification task, freezing the base model was sufficient to obtain good validation performance.

Result: Feature extraction was preferred over fine-tuning for the subsequent cross-validation stage because it achieved the higher validation accuracy in the direct comparison.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Experimental Results

Five-fold cross-validation was performed on the shortlisted configurations to check whether the validation performance was consistent across different splits of the training data. The mean validation accuracy and standard deviation were recorded for each configuration.

Table 10: 5-Fold Cross-Validation Results

Configuration	Mean Accuracy (%)	Standard Deviation (%)
Feature Extraction	90.62	1.44
Dropout (0.25)	91.03	1.20
Batch Normalization	90.83	1.14
Fine-Tuning	77.89	1.58

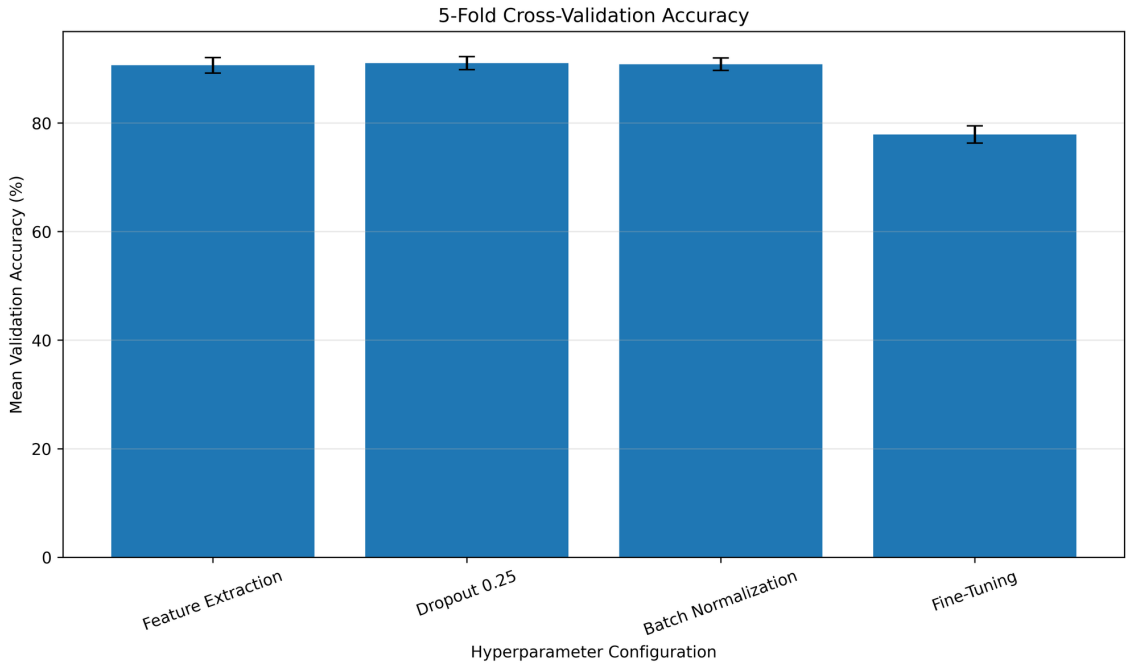


Figure 13: 5-Fold Cross-Validation Accuracy Comparison

Observation: Dropout with a rate of 0.25 achieved the highest mean validation accuracy of 91.03%. Feature Extraction and Batch Normalization also gave similar results, with mean accuracies of 90.62% and 90.83%, respectively. Fine-Tuning performed considerably worse, with a mean accuracy of 77.89%.

Inference: The standard deviations are relatively small for the three better-performing configurations, showing that their performance did not change drastically across the five folds. Dropout (0.25) also has a lower standard deviation than Feature Extraction, while Batch Normalization has the smallest variation. The much lower accuracy of fine-tuning suggests that unfreezing the pretrained layers was not beneficial with the training setup used in this experiment.

Result: Based on the highest mean cross-validation accuracy and reasonably low variation across folds, Dropout with a rate of 0.25 was selected as the final configuration for evaluation on the independent test set.

12. Final Model Evaluation

Final Test Set Results

The configuration selected through 5-fold cross-validation was Dropout with a rate of 0.25. This model was then evaluated on the independent test set, which was not used during model selection.

Table 11: Final Model Evaluation

Metric	Value
Mean CV Accuracy	91.03%
CV Standard Deviation	1.20%
Test Accuracy	89.07%
Precision	89.50%
Recall	89.01%
F1-Score	89.00%
Training Time	42.45 s
Number of Parameters	2,426,725

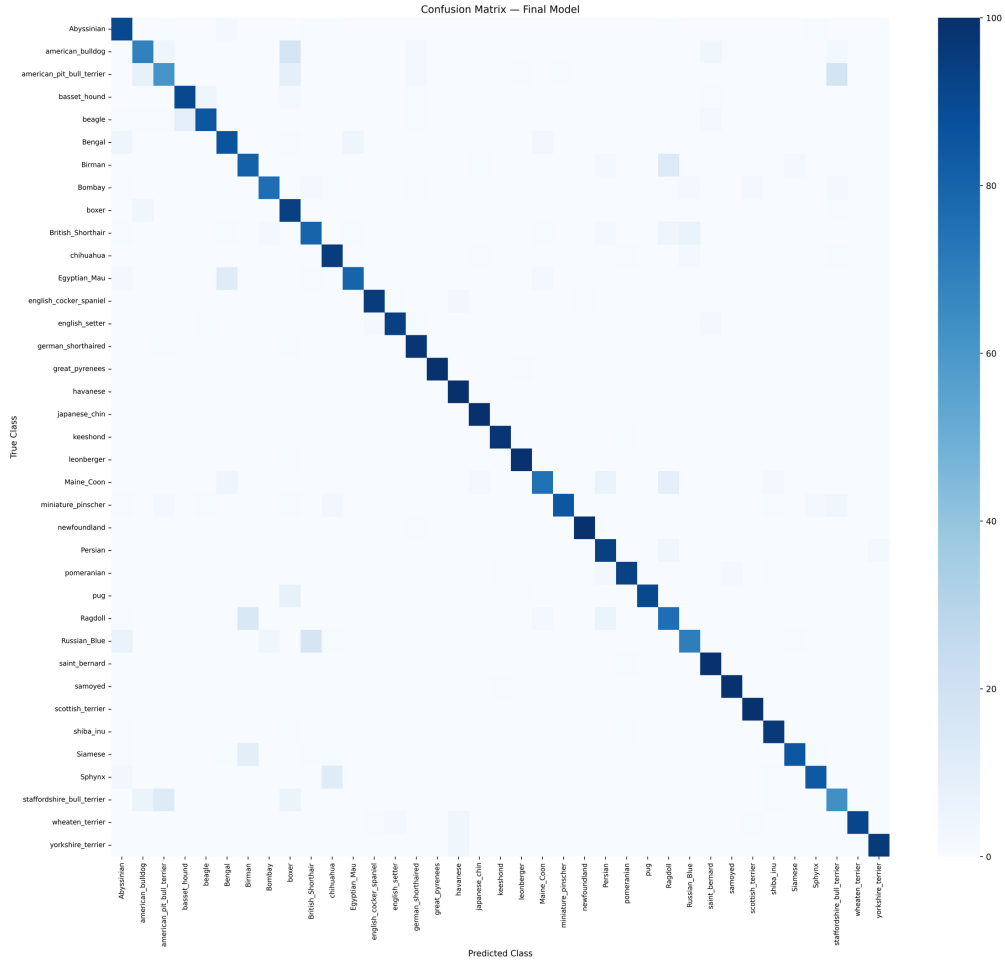


Figure 14: Confusion Matrix of the Final Model

Observation: The confusion matrix shows that the model is able to correctly classify a large number of pet-breed images, while the remaining errors are mainly due to confusion between visually similar breeds. This is expected for a fine-grained classification task where some breeds have similar visual characteristics.

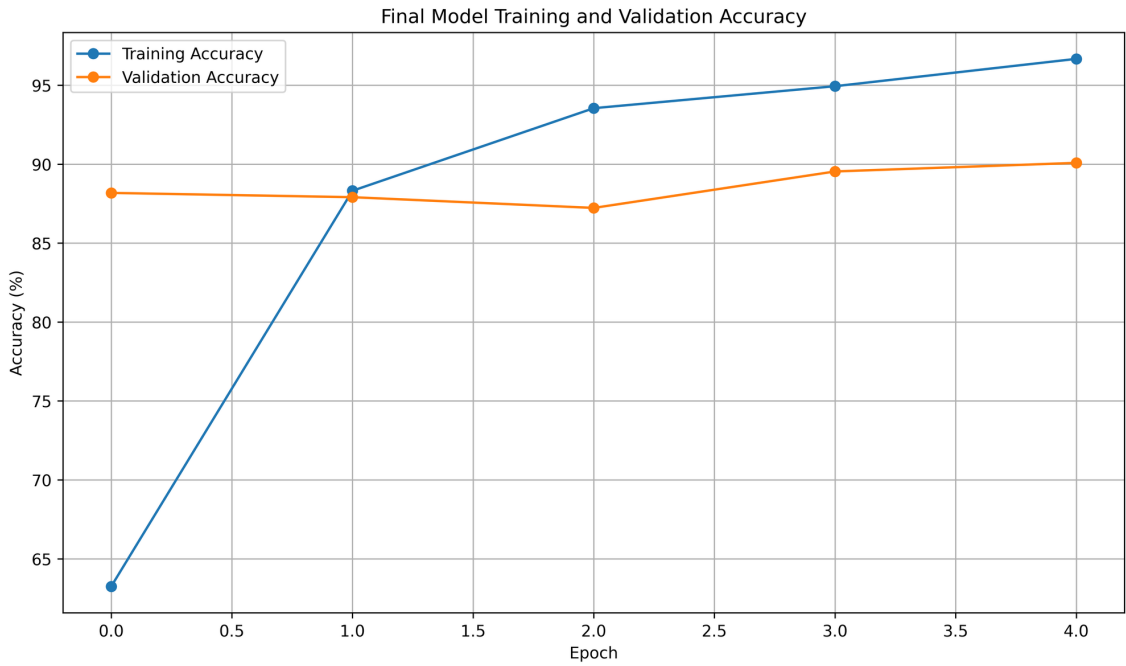


Figure 15: Final Model Accuracy

Inference: The final model achieved a test accuracy of 89.07%, with precision, recall and F1-score also close to

89%. The relatively small difference between these metrics indicates that the model’s performance is reasonably balanced rather than being strongly biased towards only a subset of the classes.

Result: The selected Dropout (0.25) configuration achieved 89.07% accuracy on the independent test set. The result confirms that the selected model generalizes reasonably well to unseen images, although some classification errors remain between visually similar pet breeds.

13. Overall Results

Table 12: Overall Experimental Results

Configuration	CV Accuracy (%)	SD (%)	Test Accuracy (%)	Training Time (s)
Baseline MobileNetV2	–	–	–	52.72
Best Initialization (He)	–	–	–	41.91
Best Regularization (Batch Normalization)	90.83	1.14	–	42.66
Best Optimizer (Adam)	–	–	–	42.81
Best Hyperparameters (Dropout = 0.25)	91.03	1.20	89.07	42.45
Fine-Tuned Model	77.89	1.58	–	46.17

Observation: The experiments show that relatively small changes in the training setup affected the validation performance. He initialization gave the highest single validation accuracy of 91.58%, while the learning rate of 0.001, Batch Normalization and Dropout of 0.25 also gave validation accuracies above 91%.

The 5-fold cross-validation results provided a more reliable comparison of the shortlisted configurations. Dropout with a rate of 0.25 achieved the highest mean accuracy of 91.03% with a standard deviation of 1.20%, and was therefore selected for the final model.

Final Test Result: The selected model achieved an accuracy of 89.07% on the independent test set, with a precision of 89.50%, recall of 89.01% and F1-score of 89.00%.

14. Required Inference for Plots

Plot 1: Training Loss vs. Epoch – Weight Initialization

The training loss decreases across the epochs for all the initialization methods. The curves follow a similar overall trend, showing that the pretrained MobileNetV2 features allow the models to learn effectively with the different initialization methods. The differences between the methods are relatively small.

Plot 2: Validation Accuracy vs. Epoch – Weight Initialization

Validation accuracy improves during the initial epochs and then starts to level off. He initialization gives the highest validation accuracy of 91.58%, although there is a small decrease in the final epoch. Overall, He initialization gives the best validation performance among the four methods tested.

Plot 3: Training and Validation Accuracy – Regularization

Training accuracy increases for all the regularization methods, but the validation accuracy improves at a slower rate. The difference between training and validation performance indicates a generalization gap. Batch Normalization gives the highest validation accuracy among the compared methods.

Plot 4: Training and Validation Loss – Regularization

Training loss decreases throughout the experiment for all the configurations. The model without regularization reaches a very low training loss while its validation loss remains higher, indicating some overfitting. The regularized models maintain a better balance between fitting the training data and generalizing to validation data.

Plot 5: Validation Accuracy With and Without Batch Normalization

The model with Batch Normalization achieves a higher validation accuracy than the model without Batch Normalization. The improvement from 90.49% to 91.17% shows that Batch Normalization helped the model learn more effectively in this experiment.

Plot 6: Training Loss vs. Epoch – Optimizers

The training loss decreases considerably faster for Momentum, RMSProp and Adam than for SGD. SGD remains at a much higher loss throughout the training period, while Adam and RMSProp reach much lower losses by the later epochs.

Plot 7: Validation Accuracy vs. Epoch – Optimizers

Adam achieves the highest validation accuracy of 90.49%, followed by RMSProp and Momentum. SGD performs considerably worse and reaches only 23.37%. This shows that the adaptive optimization methods were more effective for the given training setup.

Plot 8: Learning Rate vs. Validation Accuracy

The learning rate of 0.001 performs better than 0.0001. It reaches 91.44% validation accuracy, while the smaller learning rate reaches 87.64%. The lower learning rate appears to learn more slowly within the five epochs used.

Plot 9: Batch Size vs. Validation Accuracy

The validation accuracy improves slightly with increasing batch size. Batch size 64 gives the highest validation accuracy of 91.30%, while batch sizes 16 and 32 give 90.62% and 90.90%, respectively. The difference is relatively small, showing that batch size has a smaller effect than some of the other parameters tested.

Plot 10: Dropout Rate vs. Validation Accuracy

A dropout rate of 0.25 gives the best validation accuracy of 91.30%. Using no dropout gives a lower accuracy of 89.67%, while increasing the dropout rate to 0.5 reduces the accuracy to 90.22%. This suggests that a moderate amount of dropout provided the best balance for this model.

Plot 11: Feature Extraction vs. Fine-Tuning

Feature extraction performs slightly better than fine-tuning in the given experiment. The validation accuracy reaches 90.49% with feature extraction compared with 89.95% with fine-tuning. Therefore, unfreezing the pretrained layers did not provide an improvement under the training conditions used.

Plot 12: Training and Validation Loss – Transfer Learning

Both approaches show a reduction in training loss as the epochs progress. Feature extraction maintains slightly better validation behaviour than fine-tuning. Since the pretrained features were already useful for the task, allowing the base layers to update did not give an additional benefit in this experiment.

Plot 13: 5-Fold Cross-Validation

The cross-validation results show that Dropout with a rate of 0.25 has the highest mean accuracy of 91.03%. Batch Normalization has the lowest standard deviation of 1.14%, indicating slightly more consistent performance across the folds. Fine-tuning performs much lower than the other configurations.

Plot 14: Confusion Matrix

Observation: The confusion matrix shows that most of the predictions lie along the diagonal, indicating that the model correctly classifies most of the pet breeds. Classes such as Boxer, Chihuahua, German Shorthaired, Great Pyrenees, Japanese Chin, Keeshond, Leonberger and Samoyed show particularly strong diagonal predictions. Some noticeable confusion can be seen between American Pit Bull Terrier and American Bulldog, Basset Hound and American Pit Bull Terrier, and a few other breeds with weaker off-diagonal predictions. These errors can occur because of similarities in body shape, facial features, pose, fur patterns and variations in the images.

Plot 15: Final Model Accuracy

The final selected model achieves 89.07% accuracy on the independent test set. Its precision, recall and F1-score are also close to 89%, indicating that the model maintains reasonably balanced performance across the classification task.

15. Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Model parameters are values learned automatically by the model during training, such as weights and biases. Hyperparameters are values chosen before training, such as learning rate, batch size, dropout rate and number of epochs. Parameters are updated during training, whereas hyperparameters control how the training process takes place.

2. Why is weight initialization important?

Weight initialization determines the starting values of the trainable weights. A suitable initialization helps the network learn effectively and can improve convergence while reducing problems such as vanishing or exploding activations.

3. Why can zero initialization be problematic for neural networks?

If all weights are initialized to zero, neurons in the same layer receive the same gradients and continue to learn the same features. This breaks the symmetry between neurons and prevents them from learning different representations effectively.

4. Compare Xavier and He initialization.

Xavier/Glorot initialization is mainly designed to keep the variance of activations stable and is commonly associated with sigmoid or tanh activations. He initialization uses a variance suited to ReLU-type activations and is generally more appropriate for networks using ReLU or similar activations. In this experiment, He initialization gave the highest validation accuracy among the initialization methods tested.

5. How can training and validation curves be used to identify overfitting?

Overfitting can be identified when training performance continues to improve while validation performance stops improving or becomes worse. Similarly, if training loss keeps decreasing while validation loss starts increasing, the model is beginning to fit the training data too closely.

6. How does Dropout reduce overfitting?

Dropout randomly deactivates a fraction of neurons during training. This prevents the network from depending too much on particular neurons and encourages it to learn more general features, which can improve its performance on unseen data.

7. What is the purpose of Batch Normalization?

Batch Normalization normalizes activations within a mini-batch during training. It helps maintain more stable activation values and can make training more consistent and efficient. The learnable parameters allow the network to adjust the normalized values when required.

8. Explain the numerical Batch Normalization example.

For the input values $x = [2, 4, 6, 8]$, the batch mean is 5 and the batch variance is 5. Therefore, the standard deviation is approximately 2.236. The values are then normalized using the mean and variance, giving approximately $[-1.342, -0.447, 0.447, 1.342]$ when ϵ is ignored. If $\gamma = 1$ and $\beta = 0$, these normalized values become the final outputs. This produces approximately zero-centred and unit-scaled activations for the mini-batch.

9. What are the roles of γ and β ?

γ and β are learnable parameters in Batch Normalization. γ controls the scale of the normalized activation, while β controls its shift. This allows the network to learn the most suitable scale and centre for the activations.

10. Compare SGD, Momentum, RMSProp and Adam.

SGD updates the weights directly using the current gradient and can converge slowly. Momentum uses information from previous updates to reduce oscillations and speed up convergence. RMSProp adapts the learning rate for individual parameters based on recent squared gradients. Adam combines momentum-like updates with adaptive learning rates. In this experiment, Adam achieved the highest validation accuracy among the four optimizers.

11. What happens when the learning rate is too large?

A very large learning rate can cause the optimizer to overshoot the minimum instead of approaching it smoothly. The loss may fluctuate or even diverge, resulting in unstable training and poor performance.

12. What happens when the learning rate is too small?

A very small learning rate makes the weight updates very small. As a result, training becomes slow and the model may not reach a good solution within the available number of epochs.

13. What is the effect of increasing batch size?

Increasing the batch size means that more training samples are used for each weight update. This generally makes the gradient estimate less noisy and can improve computational efficiency, but it also requires more memory. In this experiment, batch size 64 gave the highest validation accuracy among the tested batch sizes.

14. Explain stride and padding.

Stride is the number of pixels by which a convolution filter moves across the input. A larger stride reduces the spatial dimensions more quickly. Padding adds extra values, usually zeros, around the border of the input. It can be used to control the output size and preserve information near the edges.

15. Why is MobileNetV2 computationally efficient?

MobileNetV2 is designed to reduce the number of computations and parameters while maintaining useful image representations. It uses depthwise separable convolutions, inverted residual blocks and linear bottlenecks, making it suitable for resource-constrained environments such as CPU-based laboratory experiments.

16. What is depthwise separable convolution?

Depthwise separable convolution separates a standard convolution into two operations: depthwise convolution and pointwise 1×1 convolution. Depthwise convolution applies a filter independently to each input channel, while pointwise convolution combines the resulting channels. This significantly reduces the number of computations compared with a standard convolution.

17. What is transfer learning?

Transfer learning is the process of using knowledge learned by a model on a large dataset for a different but related task. In this experiment, the MobileNetV2 model pretrained on ImageNet was used as the starting point for pet-breed classification.

18. Differentiate feature extraction and fine-tuning.

In feature extraction, the pretrained convolutional base is kept frozen and only the newly added classifier is trained. In fine-tuning, some of the pretrained layers are unfrozen and trained along with the new classifier so that the learned features can be adapted to the target dataset.

19. Why is a smaller learning rate generally used during fine-tuning?

The pretrained network already contains useful features. Using a large learning rate could change these weights too quickly and destroy the useful representations learned during pretraining. A smaller learning rate allows the pretrained features to be adjusted gradually to the new task.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

K-Fold Cross-Validation evaluates a configuration on several different training and validation splits. This gives a better indication of how consistently the configuration performs instead of relying on only one split. It therefore provides a more reliable basis for comparing hyperparameters.

21. Why must the test set remain untouched during tuning?

The test set should represent completely unseen data. If it is used during hyperparameter tuning, information from the test set can influence model selection, making the final test performance overly optimistic. Therefore, it should only be used after the final configuration has been selected.

22. Why should mean and standard deviation both be reported?

The mean shows the average performance of a configuration across the folds, while the standard deviation shows how much that performance varies between folds. Reporting both gives information about accuracy as well as consistency and reliability.

23. Is the highest validation accuracy always sufficient to select a model?

No. Validation accuracy alone may not be enough to select a model. Cross-validation variability, generalization to the independent test set, training time and computational cost should also be considered. A model with slightly lower accuracy but much more consistent performance may be a better choice.

16. Additional Exercise

Two new combinations of learning rate, dropout rate, batch size and fine-tuning strategy were tested using 5-fold cross-validation. The results were compared with the selected configuration, Dropout 0.25, which had a mean CV accuracy of 91.03% with a standard deviation of 1.20%.

Table 13: Configurations used for the Additional Exercise

Configuration	Learning Rate	Dropout	Batch Size	Fine-Tuning
Selected Configuration	0.001	0.25	64	No
Additional C1	0.0001	0.50	16	No
Additional C2	0.00001	0.50	64	Yes

Table 14: 5-Fold Cross-Validation Results for Additional Configurations

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD (%)	Time (s)
Selected Configuration	–	–	–	–	–	91.03 $\pm$ 1.20	–
Additional C1	89.81	87.95	88.79	86.59	84.35	87.50 $\pm$ 1.90	40.98
Additional C2	55.18	63.67	53.82	53.65	59.01	57.07 $\pm$ 3.82	44.40

Observation: Additional C1 performed reasonably well, with a mean accuracy of 87.50%, but it was still lower than the selected configuration at 91.03%. Its standard deviation of 1.90% also shows slightly more variation across the folds. Additional C2 performed considerably worse, with a mean accuracy of only 57.07% and a higher standard deviation of 3.82%.

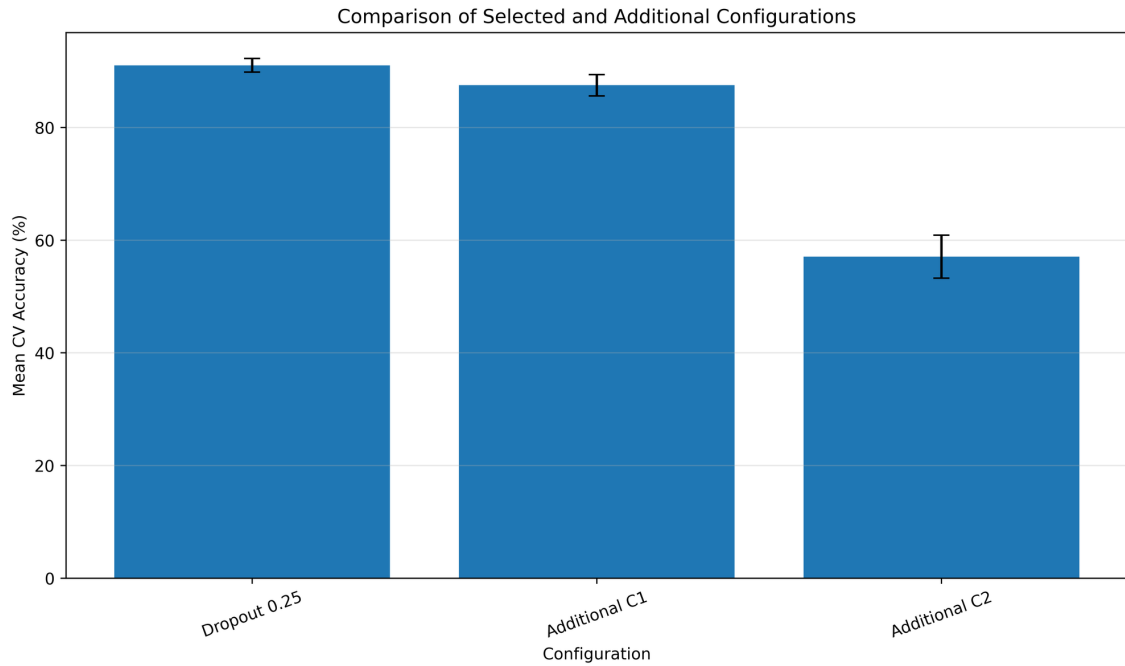


Figure 16: Comparison of Selected and Additional Configurations

Inference: The selected configuration gives the highest and most consistent validation performance among the three configurations. Additional C1 is reasonably close but does not improve the result, while Additional C2 performs much worse. The lower learning rate combined with fine-tuning in Additional C2 did not provide enough improvement within the five epochs used.

Result: Neither of the additional configurations was preferable to the selected Dropout 0.25 configuration. Additional C1 had a lower mean accuracy despite a comparable training time, while Additional C2 had both much lower accuracy and greater variation. Therefore, the originally selected configuration remains the better choice based on validation accuracy, consistency and computational cost.

17. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, MobileNetV2: Inverted Residuals and Linear Bottlenecks, CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, Cats and Dogs, IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>